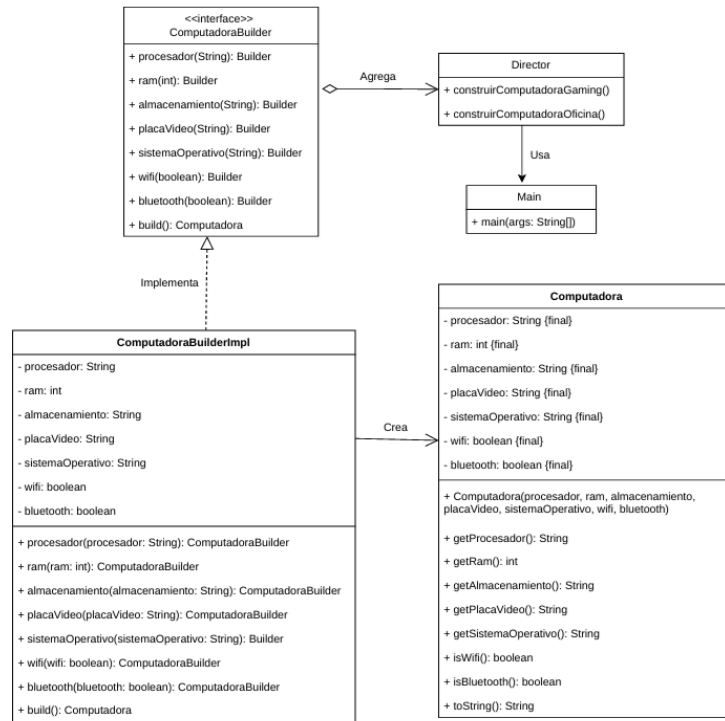


Trabajo Práctico: Patrones de Diseño I

Patrón de Diseño: **Builder**

1. Diagrama UML de clases

a. Diagrama UML Builder



b. Descripción de Elementos

- **Computadora (Producto):** Representa el objeto complejo que se desea construir. Sus atributos son de tipo final para garantizar la inmutabilidad una vez que la computadora ha sido totalmente ensamblada.
- **ComputadoraBuilder (Interfaz Builder):** Define los métodos fluidos de configuración paso a paso, tales como procesador, ram, almacenamiento, placa video, sistema operativo, wifi y bluetooth, junto con el método final build que retorna el objeto listo.
- **ComputadoraBuilderImpl (Builder Concreto):** Implementa la interfaz ComputadoraBuilder. Almacena temporalmente los valores de cada parte y finalmente instancia el objeto Computadora pasándole los parámetros acumulados.
- **Director:** Contiene métodos predefinidos como construirComputadoraGaming y construirComputadoraOficina, los cuales conocen el orden y la receta exacta para fabricar configuraciones específicas utilizando el Builder.
- **Main:** Es la clase cliente que pone en marcha el sistema utilizando el Director y el Builder.
- En cuanto a las relaciones del diagrama, la clase ComputadoraBuilderImpl implementa (relación de realización) la interfaz ComputadoraBuilder. Por su parte, la clase Director tiene una relación de asociación o agregación con la interfaz ComputadoraBuilder para indicar los pasos a seguir. Finalmente, el Builder Concreto crea el objeto Computadora mediante el método de construcción.

2. Análisis del patrón

- a. **Problema:** Cuando se requiere instanciar un objeto complejo que posee múltiples atributos y configuraciones opcionales u obligatorias (como una Computadora con procesador, memoria RAM, almacenamiento, placa gráfica, etc.), el uso de constructores tradicionales extensos genera código difícil de leer, propenso a errores al confundir el orden de los parámetros y poco flexible.
 - b. **Solución:** El patrón Builder separa la construcción de un objeto complejo de su representación. Delega la creación paso a paso en un objeto especializado y permite encapsular recetas de ensamblaje en una clase Director, logrando que un mismo proceso de construcción pueda generar diferentes configuraciones de objetos de manera limpia y legible mediante una interfaz fluida.
 - c. **Consecuencias:** Entre las
 - Ventajas se destacan el control del flujo de construcción paso a paso, la favorabilidad hacia la inmutabilidad del producto final mediante atributos de tipo final, la prevención de constructores sobrecargados gigantes y la mejora sustancial en la legibilidad del código cliente.
 - Desventaja incrementa ligeramente la cantidad de clases e interfaces accesorias en el sistema.
3. implementación
- a. Código implementado
 - b. Breve explicación del funcionamiento
 - La solución implementa el patrón de diseño Builder, cuyo objetivo es separar la construcción de un objeto complejo de su representación, permitiendo crear un objeto paso a paso de forma limpia y controlada.
 - En este caso, la clase Computadora representa el producto complejo que se desea construir, conteniendo múltiples atributos de tipo final (procesador, ram, almacenamiento, placaVideo, sistemaOperativo, wifi, bluetooth) para garantizar que una vez armada, sus datos no se modifiquen.
 - Para gestionar la construcción paso a paso, el sistema utiliza una interfaz llamada ComputadoraBuilder, la cual define las firmas de los métodos de configuración necesarios (como procesador(), ram(), etc.) y el método final build().
 - La clase ComputadoraBuilderImpl implementa dicha interfaz, actuando como el builder concreto que almacena temporalmente los valores de cada parte a medida que se van asignando. Cada método de configuración devuelve this para permitir llamadas encadenadas (*fluent interface*).
 - El proceso de ensamblaje se apoya en una clase Director, la cual contiene métodos predefinidos como construirComputadoraGaming() y construirComputadoraOficina(), encargados de conocer el orden y la receta exacta para fabricar configuraciones específicas utilizando el builder.
 - Por ejemplo, cuando la clase cliente (Main) ejecuta: Computadora computadoraGaming = director.construirComputadoraGaming(new ComputadoraBuilderImpl()); El Director invoca secuencialmente los métodos del builder para configurar el procesador, la memoria RAM, la placa de video y demás componentes con los valores óptimos para juegos.
 - Finalmente, al invocar el método build(), el builder concreto instancia y devuelve el objeto Computadora completamente armado y listo para ser utilizado.
 - Esto permite evitar constructores sobrecargados gigantescos o confusos, facilitando la creación de distintas representaciones de un producto de manera flexible, legible y desacoplada.
4. Principios de patrón Builder

- **Principio de Responsabilidad Única (SRP):** La clase Computadora se limita exclusivamente a mantener el estado de sus componentes inmutables. La lógica de ensamblaje recae sobre el Builder y la coordinación de recetas sobre el Director.
- **Principio Abierto/Cerrado (OCP):** Si en el futuro se requiere agregar nuevos tipos de computadoras, es posible extender el Director o crear nuevos métodos sin modificar las clases base existentes.
- **Encapsulamiento y Ocultamiento de Información:** Los atributos de la computadora son de tipo final y carecen de métodos modificadores (setters), garantizando que el objeto no pueda ser alterado incorrectamente una vez construido.

5. Principios SOLID cómo se relaciona

- **S - Single Responsibility Principle**
 - La clase Computadora solo se encarga de almacenar y representar el estado final de los datos (es una clase de modelo inmutable).
 - La clase ComputadoraBuilderImpl se encarga exclusivamente de la lógica de construcción y ensamblaje paso a paso.
 - La clase Director se encarga únicamente de coordinar las recetas o configuraciones predefinidas (como gaming u oficina).
- **O - Open/Closed Principle**
 - Si mañana se necesita crear una nueva variante de computadora (por ejemplo, una "Computadora de Desarrollo"), no necesitas modificar la clase Computadora ni la interfaz del builder. Simplemente puedes agregar un nuevo método en el Director o crear un nuevo builder concreto, extendiendo el sistema sin romper lo que ya funciona.
- **D - Dependency Inversion Principle**
 - En la clase Director, los métodos reciben como parámetro la interfaz ComputadoraBuilder y no la implementación concreta (ComputadoraBuilderImpl). Esto significa que el Director está desacoplado de los detalles de cómo se construye el objeto; solo le importa que cumpla con la interfaz.